

UNIVERSITY OF KENT

SCHOOL OF COMPUTING

Contextual Word Replacement System with Transformer-Based Language Models

MASTER THESIS

Ugur Doga Sezgin

Supervisor	Prof. Frank Wang
Student Name	Ugur Doga Sezgin
Student Number	25040132
Student Email	us63@kent.ac.uk
Field of Study	Artificial Intelligence
Submission Date	2025-09-02
Word Count	9265

Abstract

Finding alternatives to words given in context can be helpful in use cases such as language learning and writing assistance. This task of identifying suitable replacements is called Lexical Substitution (LS). This thesis explores recent LS methods and provides an open-source project implementing several transformer-based approaches.

Two model families were the focus of this work. For Masked Language Models (MLMs), a BERT-based embedding dropout method was implemented, with several variations using RoBERTa and different ways to inject the target word. For Causal Language Models (CLMs), GPT-2, GPT-4o and LLaMa were explored for both candidate generation and ranking. For analysis and comparison, a benchmarking tool was developed using common LS datasets and metrics.

Overall, GPT-4o produced the highest benchmark scores, but it does not provide deterministic candidate scoring. For the candidate ranking task, Llama-based sentence similarity achieved the best results. Among MLMs, RoBERTa with the dropout technique outperformed the other approaches.

Contents

1	Introduction	1
2	Literature Review	4
2.1	Early Methods	4
2.1.1	Knowledge-Based Methods	4
2.1.2	N-gram Language Model Methods	4
2.1.3	Count-Based Vector Methods	5
2.1.4	Supervised Methods	5
2.1.5	Embedding Similarity Methods	6
2.2	Transformer Models	7
2.2.1	BERT	8
2.2.2	RoBERTa	9
2.2.3	GPT-2	9
2.2.4	LLaMa	9
2.3	Transformer-Based Methods	10
2.3.1	BERT Dropout	10
2.3.2	BERT-LS	11
2.3.3	LexSubGen	12
2.3.4	Genesis	12
2.3.5	LexSubCon	13
2.3.6	ParaLS	13
2.3.7	PromptSub	14
3	Methodology and Implementation	15
3.1	Methodology Overview	15
3.2	Implementation Overview	16
3.3	Console Tool	17
3.4	Benchmark Reporter	18
3.5	Substituters	20
3.5.1	BERT Substituters	21
3.5.2	GPT Substituters	23
3.5.3	LLaMa Substituter	25
4	Analysis and Evaluation	28
4.1	Dropout Implementations	28
4.2	Similarity Measures	28
4.3	Masked Language Models	29
4.4	Target Injection Techniques	30
4.5	Exclusion Filters	30

4.6	CLM Methods	31
4.7	Candidate Ranking	32
5	Conclusions and Future Work	34
	Bibliography	36
A	Appendix: Sample Benchmark Report	39
B	Appendix: Sample Substitution Tables	40

1 Introduction

Lexical Substitution (LS) is the task of identifying most suitable alternatives to a target word given in context such as a sentence or paragraph. It requires a distinct approach compared to synonym selection, as the meanings of words are heavily influenced by their surrounding context. This contextual sensitivity can cause synonym candidates to alter the intended meaning or disrupt the natural flow, whereas unrelated words can sometimes integrate smoothly while preserving the original meaning.

This thesis investigates several LS methods based on pretrained language models and aims to develop an LS system with a deterministic and reliable scoring mechanism. In recent years, methods using transformer-based language models such as BERT and RoBERTa have significantly advanced the field, outperforming traditional approaches reliant on external lexical resources such as WordNet. This project implements and benchmarks various state-of-the-art techniques and explores novel approaches using decoder-only architectures, including LLaMa and GPT series.

The motivation behind this project stems from the wide range of potential applications offered by an effective lexical substitution system. For instance, such a system could facilitate quick, context-aware word lookups during reading activities, including books or subtitled content. Context-sensitive LS systems can efficiently provide accurate single-word replacements that preserve nuanced meanings. Additionally, other significant applications include interactive language-learning tools, paraphrase evaluation, and enhanced writing assistance.

The primary development challenge involves generating high-quality candidates and accurately scoring each candidate. Most LS methods divide the task into two subtasks: candidate proposal and candidate ranking. During the proposal phase, a specified number of potential replacements are generated. In the ranking phase, candidates are scored and ordered based on their suitability. The final ranked predictions, typically the top ten, are then benchmarked against human-annotated substitutions from widely used LS datasets such as SemEval 2007 (LS07) and CoinCo (LS14).

Multiple approaches have been systematically explored in this project¹ for both proposal and ranking subtasks. Initially, BERT-based embedding dropout strategy (Zhou et al., 2019) was implemented. Following detailed problem analysis and iterative refinements, this implementation achieved slightly higher benchmark scores compared to the original study. This achievement represents a valuable contribution,

¹ <https://github.com/udoga/word-replacer>

particularly given the absence of publicly available implementations of this method, despite its popularity and prominence in LS research literature.

Prior to working on alternative approaches, a benchmarking tool was developed for effective analysis and measurement. Common evaluation metrics from existing studies (best, best-mode, oot, oot-mode, precision@1, precision@3, recall@10, GAP) were implemented following their original definitions. The tool loads the selected dataset (LS07 or LS14, trial or test subsets), processes substituter predictions, and generates a detailed report including final average scores and individual predictions and scores for each dataset sample. This tool provided a basis for consistent comparisons and guided iterative improvements.

Following the initial substituter, additional techniques based on Masked Language Models (MLMs) were explored. A key consideration is how target word information is injected into the model. The approach proposed by (Zhou et al., 2019) applies partial masking (embedding dropout) to the target word to strike a balance between capturing its semantics and contextual information. Alternative injection strategies, including Duplicate Input and Dynamic Patterns, were also implemented and tested with various configurations. While these alternatives yielded comparable results, the original dropout-based method achieved slightly better performance.

In the next phase, Causal Language Models (CLMs) were explored due to their limited use in LS task despite their strong performance in various domains. GPT-2 and LLaMa were tested locally, while GPT-4o was accessed via a web API. GPT-2 used beam search with a Duplicate Input-style technique and pseudo-log-likelihood scoring, but results fell short of MLM-based approaches. In contrast, GPT-4o achieved the highest scores across all metrics by a significant margin. However, its outputs lacked deterministic candidate scoring, which was a primary objective of this project.

Finally, LLaMa model was used for experiments with a large CLM running locally. Candidate proposals were generated from the model’s response to a prompt. This raw substitution list outperformed the best MLM-based approaches across all evaluation metrics. For candidate ranking, similarity between original and candidate sentence embeddings from the model’s final hidden states was used, yielding higher GAP ranking scores than previous scoring methods. Additionally, BARTScore was tested as an alternative ranking strategy.

This thesis begins with a review of background and related work in lexical substitution, covering both traditional approaches and recent transformer-based methods. The following chapter describes the methodology and implementation of this project, detailing the design and development of multiple substituters. This is followed by an

in-depth analysis, presenting benchmark results and comparing with findings from previous studies. The final chapter discusses the limitations of the current work and suggests potential directions for future research.

2 Literature Review

The task of Lexical Substitution (LS) was first introduced by (McCarthy & Navigli, 2007), which provided both a benchmark dataset (SemEval 2007) and evaluation metrics for assessing system performance. Building on this foundation, Kremer et al. (2014) later proposed an extension of the task with the Concepts in Context (Co-inCo) dataset, offering a broader set of examples and more diverse human-generated substitutions. These two datasets have since become the standard benchmarks for the majority of LS studies.

Over time, a wide range of approaches have been developed for the two main LS subtasks: candidate generation (proposal) and candidate ranking (scoring). For candidate generation, traditional approaches mostly relied on lexical resources such as WordNet (Fellbaum, 1998), Roget’s Thesaurus, or the Encarta Encyclopedia. What distinguished these systems was their candidate ranking strategies, which gave rise to approaches described in the following section.

2.1 Early Methods

2.1.1 Knowledge-Based Methods

Knowledge-based methods rely on structured lexical resources and rule-based logic to rank substitution candidates. These methods often incorporate Word Sense Disambiguation (WSD), which aims to identify the most appropriate sense of the target word based on its context. Once a sense is selected, synonyms associated with that sense are prioritized. Another technique, known as Lexical Baseline, assigns higher scores to synonyms found in multiple lexical resources.

An example of a knowledge-based system is SubFinder (Hassan et al., 2007), which integrates multiple ranking components including Lexical Baseline, a WSD tool called SenseLearner (Mihalcea & Faruque, 2004), and Google N-gram corpus for statistical n-gram modeling, which is described in the next section.

2.1.2 N-gram Language Model Methods

N-gram language models (LM) assign a probability to a word sequence, where the likelihood of each word depends only on the previous $n - 1$ tokens. This is known as the $(n-1)$ -order Markov assumption. For a trigram model ($n = 3$), for example, the probability of the sentence is approximated as:

$$P(w_1, \dots, w_T) = P(w_1) \prod_{t=2}^T P(w_t \mid w_{t-2}, w_{t-1}) \quad (2.1)$$

Each conditional probability is estimated from corpus counts and smoothed so that unseen n-grams receive non-zero mass. For example, before smoothing, a trigram model finds the probability of each word by using the following formula, where C is the count (number of appearances) of the n-gram from the corpus:

$$P(w_3 \mid w_1, w_2) = \frac{C(w_1 w_2 w_3)}{C(w_1 w_2)} \quad (2.2)$$

For candidate ranking, LS methods often replace the target word with each candidate and compute the resulting sentence’s probability using n-gram LM. A notable example is the system by (Yuret, 2007), which builds a 5-gram LM from the Web1T corpus to rank substitution candidates. The author of this study later proposed FASTSUBS (Yuret, 2012), which finds most probable candidates based on LMs, by using an efficient search algorithm skipping large portions of the vocabulary.

2.1.3 Count-Based Vector Methods

Distributional (count-based) vector models turn every lemma in the vocabulary into a sparse vector containing how often it appears near other lemmas within a fixed-size window. Resulting sparse vectors form a co-occurrence matrix, where rows and columns correspond to vocabulary lemmas. Some variants use a dependency-based vector space, which also considers the neighbour’s relation to the target word. The values in the matrix can be raw co-occurrence counts, or Pointwise Mutual Information (PMI) values which provide an alternative association measure.

Methods such as (Erk & Padó, 2008) and (Dinu & Lapata, 2010) bias the target word vector with its context. For candidate ranking, cosine similarity between this context-adapted target vector and plain candidate vector was used.

2.1.4 Supervised Methods

Supervised methods frame the LS task as a machine learning problem. Each training example typically consists of information about the context sentence, the target word, and one candidate substitute; followed by a binary label indicating whether the given substitute is valid or not in this context. After the training, binary classifier models learn to produce a probability score for new candidates given in new contexts.

Early supervised approaches such as (Biemann, 2013) train a separate sense classifier (WSD model) for each target word. This method relies heavily on large numbers of sense-tagged examples per word. During lexical substitution, it uses lexicalized features such as the lemma of the target word to disambiguate its meaning.

To address the limitations of lexicalized features, (Szarvas et al., 2013) proposed a global (all-words) supervised model which uses delexicalized (non-lexical) features that are independent from words themselves. Their delexicalized features included n-gram frequencies of the substitute within context, and distributional similarity score between the substitute and the target word. This approach allowed the model to generalize beyond target words seen in the training data.

Building on the idea of delexicalized features, (Hintz & Biemann, 2016) incorporated additional unsupervised word embedding features (Melamud et al., 2015). To turn these features into a ranked list of substitutes, they employed the LambdaMART model (Wu et al., 2010), which is a gradient-boosted tree ensemble optimized for ranking tasks. Their model was multilingual, and achieved state-of-the-art performance on both the “best” and “oot” measures.

2.1.5 Embedding Similarity Methods

Word/context embedding similarity methods marked a significant transition in LS research, moving away from sparse vectors towards dense, low-dimensional representations learned by neural networks.

One influential word embedding technique is skip-gram, which learns word embeddings by taking a center word as an input, and trying to predict surrounding words in a small window. The opposite of this approach is CBOW, where the center word is predicted from its bag-of-words context. These two are the main techniques implemented inside the word2vec toolkit. In both methods, two embeddings are learned for each word: one for when the word is the target and one for when it is one of the surrounding (context) words.

Usually the context-purposed word embeddings are treated as internal and discarded after training, but (Melamud et al., 2015) shows that these embeddings are in fact useful for lexical substitution. Their skip-gram based approach combines (i) semantic similarity between the candidate and the target word (measured with the two words’ target vectors) and (ii) context compatibility of the candidate (measured with the candidate’s target vector and the context vectors of the surrounding words). Four scoring variants (Add, BalAdd, Mult, BalMult) differ only in how they aggregate

these two components. This study presented a simple, easy-to-implement LS method that achieved state-of-the-art results among unsupervised approaches.

In their later study, (Melamud et al., 2016) introduced context2vec, a model and toolkit that learns a generic context embedding. Unlike the previously used skip-gram approach, this model is based on the CBOW architecture. In standard CBOW, the embeddings of the surrounding words are averaged to form a context vector. This study replaces this averaging operation with a bidirectional LSTM, which yields a more accurate context representation. For the LS task, candidates are ranked by the cosine similarity between their target embedding and the context embedding produced by the bidirectional LSTM. GAP evaluation results indicate significantly better candidate ranking than the baseline that uses simple averaging.

These early neural network-based methods reframed substitution as finding the closest word and context embeddings in a continuous semantic space. While effective, they still lacked mechanisms such as self-attention, a shortcoming later addressed by Transformer-based architectures.

2.2 Transformer Models

Transformers (Vaswani et al., 2017) revolutionized natural language processing by allowing language models to process tokens and compute their relationships in parallel. This architecture overcame the limitations of RNN/LSTM, such as sequential processing and difficulty in capturing long-range dependencies.

Transformers are built on the self-attention mechanism, which produces new representations for each token based on the entire sequence. One self-attention head can be thought of as an attention matrix, which, when multiplied by the input embedding matrix, produces the output matrix. To achieve this, every token is first turned into three vectors (query, key and value) by multiplying its input embedding with learnable weights. Next, the query and key matrices are multiplied to score how much attention each token gives to every other token. This score matrix is scaled and softmax is applied so the scores become weights that sum to 1. Finally, this weight matrix is multiplied by the value matrix to produce new, context-aware embeddings.

A transformer network is built by stacking multiple identical layers. Each transformer layer can be an encoder or decoder block, depending on the architecture. A typical encoder block consists of: multi-head self-attention (parallel attention blocks), a residual connection and normalisation, a feed-forward neural network, and another residual connection and normalisation. A decoder block (in decoder-only models) has mostly the same structure, with the key difference being masked self-attention, which

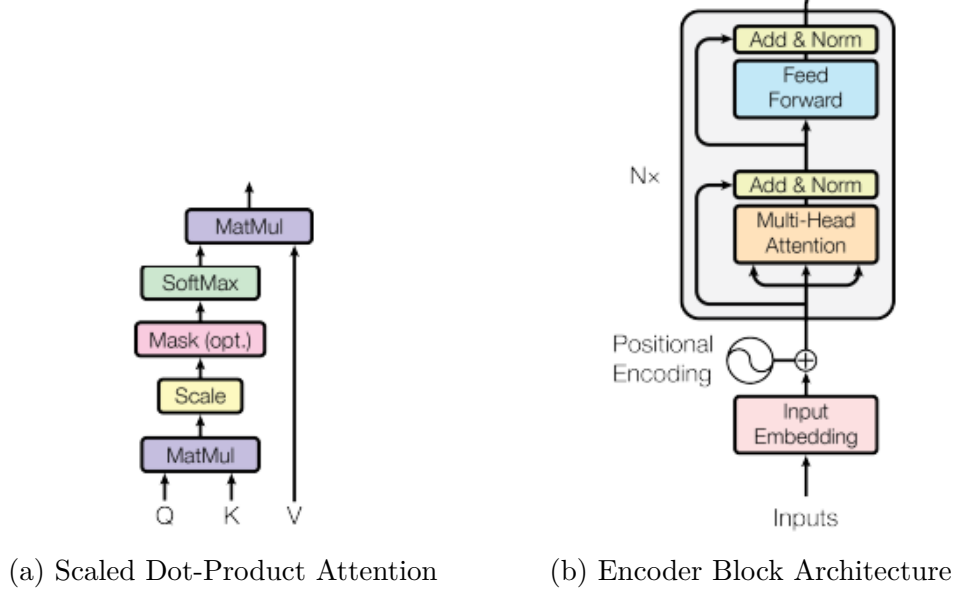


Figure 1 Attention and encoder block.

prevents tokens from attending to future positions in generation tasks. Multi-head attention enables the model to capture different types of contextual patterns, while positional encoding provides information about word order.

The transformer’s ability to process text in parallel and scale to large datasets has made it the backbone of today’s most advanced language models, offering the detailed context understanding needed for effective lexical substitution. The following subsections describe the main transformer-based models used in this study.

2.2.1 BERT

BERT (Devlin et al., 2019) is an encoder-only transformer model designed for deep context understanding. Unlike the preceding GPT model (Radford et al., 2018), it used a bidirectional architecture where every token is able to attend to both left and right context. This approach achieved state-of-the-art results on eleven NLP benchmarks. The model is pre-trained on large corpora using two objectives: Masked Language Modeling (MLM), where a subset of tokens are randomly masked and predicted, and Next Sentence Prediction (NSP), which determines whether two sentences follow each other. These objectives enable BERT to capture nuanced meanings, allowing it to be easily adapted to a wide range of tasks such as classification, question answering and lexical substitution.

2.2.2 RoBERTa

RoBERTa (Liu et al., 2019) uses the same encoder-only structure as BERT, but optimizes the pre-training process. It removes the Next Sentence Prediction task which was found unnecessary for performance. Additionally, BERT was using the same positions for masking the same sentence across epochs. This was replaced by dynamic masking, where different positions were masked each time a sentence was loaded. Finally, RoBERTa trains on a larger dataset from multiple sources, employs longer token sequences, and uses larger batch sizes. These adjustments led to significantly higher scores in several NLP benchmarks, demonstrating that BERT’s performance was constrained more by training setup than by architecture.

2.2.3 GPT-2

GPT was first introduced by (Radford et al., 2018) as a decoder-only transformer model using masked self-attention. It was pre-trained with a standard next-token prediction objective, where each token is predicted from earlier tokens using a left-shifted target sequence. After pre-training, the model was fine-tuned for different NLP tasks with minimal architectural changes. The follow-up work (Radford et al., 2019) presented GPT-2, which kept the same architecture but increased model parameters to 1.5B. Also, a larger and more diverse dataset was used. Unlike GPT, GPT-2 did not rely on task-specific fine-tuning, and achieved strong performance in multiple tasks in the zero-shot setting. This demonstrated that a sufficiently large causal language model could act as a general-purpose multitask learner.

2.2.4 LLaMa

LLaMA (Touvron et al., 2023) is a family of open, decoder-only language models trained on public datasets and released in sizes from 7B to 65B parameters. Unlike the early transformers like GPT and GPT-2, it uses pre-normalisation, where normalisation is applied before the attention and feed-forward sublayers to improve training stability. The feed-forward network uses a SwiGLU activation, a smoother variant of ReLU that often improves performance. For positional encoding, LLaMA replaces fixed absolute embeddings with rotary positional embeddings (RoPE), which rotates pairs of values in the query/key vectors in 2D subspaces to encode relative distances. RoPE is applied inside self-attention at every layer, whereas absolute position embeddings are added only once at the input. These choices make LLaMA a high-performing and resource-efficient alternative to larger models such as GPT-3.

2.3 Transformer-Based Methods

In this section, main LS studies focused on this project will be described. These methods utilize different type of transformer models such as MLMs (BERT), CLMs (GPT-2), seq2seq models (BART) and permutation-based models (XLNet). They also apply different scoring algorithms to rank candidates.

2.3.1 BERT Dropout

(Zhou et al., 2019) introduces an end-to-end BERT-based approach for the LS task. BERT model generates more context-appropriate candidates and overcomes the limitations of earlier methods, such as overlooking suitable candidates due to their reliance on lexical resources.

Researchers observed that fully masking the target word leads BERT to focus only on the context and to predict candidates unrelated to the target word. Leaving it unmasked, however, causes the model to assign most of the probability mass to the original word, making the probabilities of other candidates unreliable. For balanced consideration of both target word and its context, they propose an embedding dropout (partial masking) technique, which sets some values in the target embedding as zero.

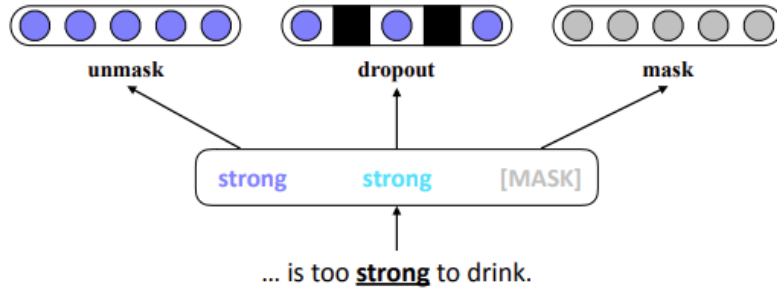


Figure 2 Embedding dropout technique compared to masking and unmasking.

Although effective, this technique introduces randomness due to the random selection of positions within the embedding. To reduce this effect, the authors repeat the experiments five times and report the average scores. This method is used in the proposal step and the top fifty predictions are used as candidates.

After candidate generation, they combine two types of scores for candidate ranking: proposal score and validation score. The proposal score (s_p) is based on the model's output probability (P) for the candidate token (x'_k) after excluding the probability for

the original target word (x_k); given the original sentence $\mathbf{x}$ and the partially masked version $\tilde{\mathbf{x}}$ at the target position k :

$$s_p(x'_k | \mathbf{x}, k) = \log \frac{P(x'_k | \tilde{\mathbf{x}}, k)}{1 - P(x_k | \tilde{\mathbf{x}}, k)} \quad (2.3)$$

The second score, candidate validation (s_v), aims to preserve the sentence meaning by finding the similarity (SIM) between the original ($\mathbf{x}$) and the candidate ($\mathbf{x}'$) sentence:

$$s_v(x'_k | \mathbf{x}, k) = \text{SIM}(\mathbf{x}, \mathbf{x}'; k) \quad (2.4)$$

To measure this similarity, for each corresponding token pair (x_i, x'_i) between the two sentences, the cosine similarity (Λ) of their contextual representation (h) is calculated. Each token's contextual representation is obtained by concatenating the internal embeddings from the last four layers, when the sentence is given to the model. These representation similarities are weighed by the average attention matrix (w) among all attention matrices from all heads and all layers in the model.

$$\text{SIM}(\mathbf{x}, \mathbf{x}'; k) = \sum_i^L w_{i,k} \times \Lambda(h(x_i | \mathbf{x}), h(x'_i | \mathbf{x}')) \quad (2.5)$$

Finally, the two scores are combined with the proposal weight (α):

$$s(x'_k | \mathbf{x}, k) = s_v(x'_k | \mathbf{x}, k) + \alpha \times s_p(x'_k | \mathbf{x}, k) \quad (2.6)$$

This method outperformed the previous LS approaches by a significant margin. However, due to lack of a publicly available implementation, later studies (Arefyev et al., 2022; Lacerra et al., 2021; Michalopoulos et al., 2021) could not reproduce the results. This thesis provides an implementation with slightly higher benchmark scores compared to the original study.

2.3.2 BERT-LS

BERT-LS (Qiang et al., 2020) targets Lexical Simplification (not substitution) which is a similar task aiming to replace a complex word with a simpler alternative. Rather than applying dropout, it injects the target word information by concatenating the original sentence and its masked variant, which is also called the Duplicate Input technique. A multi-feature ranker (BERT probability, context likelihood, semantic similarity, frequency) selects the best simplification, achieving state-of-the-art results on three benchmarks.

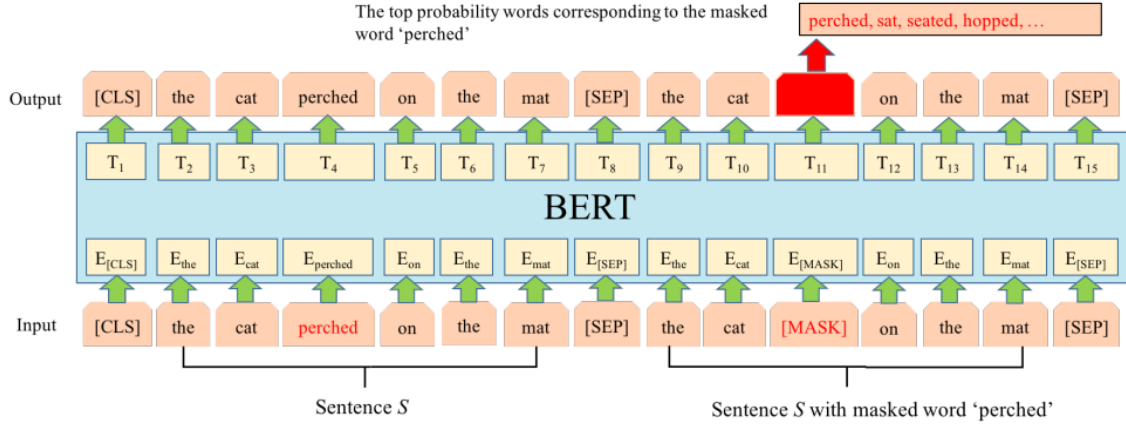


Figure 3 Duplicate Input technique.

2.3.3 LexSubGen

The study by (Arefyev et al., 2020, 2022) provides a comparison for various models (context2vec, ELMo, BERT, RoBERTa, XLNet) each with several target word injection techniques. One of these techniques is **Duplicate Input**, which duplicates the sentence and masks the target in the second sentence. Another injection technique is **Dynamic Patterns**, where the target word T is replaced with a pattern like " T or $\langle \text{MASK} \rangle$ ". They propose a new injection (also scoring) method named **+embs**, which combines the model's probability for that substitute when only the context is given $P(s|C)$, with the input embedding similarity of substitute and target $P(s|T)$. In order to prevent the dominance of $P(s|C)$ over $P(s|T)$ due to its sharper distribution, they apply a temperature parameter. They find that XLNet model combined with +embs technique outperforms the other LS methods.

2.3.4 Genesis

GENESIS (Lacerra et al., 2021) is the first generative LS system. A seq2seq model named BART is used, which is an encoder-decoder transformer where each decoder layer attends to the final encoder output. Initially, the pre-trained BART model is fine-tuned with the gold substitution datasets (CoInCo and TWSI). At inference, this model produces candidates via beam-search decoding. Generated tokens are filtered using a WordNet-based vocabulary built per target. Filtered candidates are then ranked by the cosine similarity of contextual representations in BERT (averaged over the last four layers) between the original and candidate token in context. The full system is then run on a sense-annotated corpus (SemCor) to build a silver dataset (GenSemCor). Fine-tuning BART using this silver dataset further improves results.

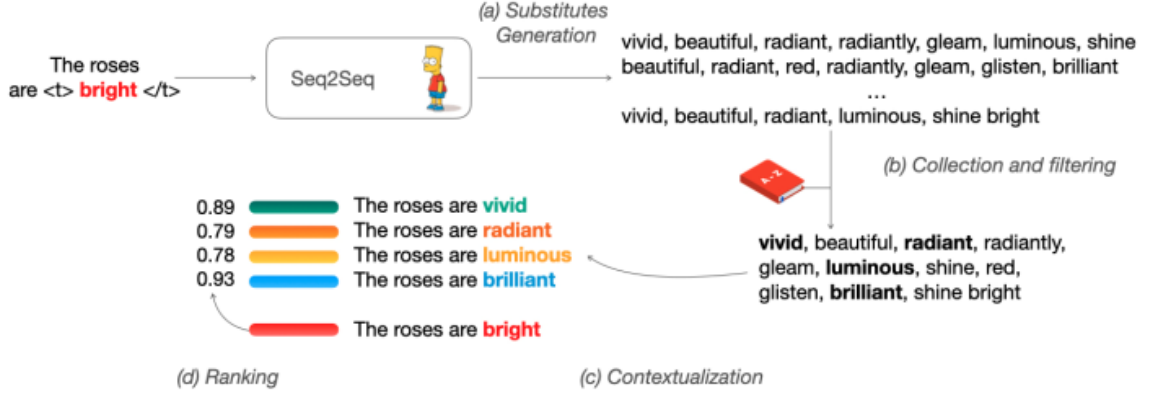


Figure 4 GENESIS pipeline.

2.3.5 LexSubCon

LexSubCon (Michalopoulos et al., 2021) integrates a BERT model with knowledge from lexical resources. For candidate generation, the target embedding is updated by linearly combining it with the mean embedding of its WordNet synonyms. Predictions from BERT are then combined with candidates from WordNet. This merged candidate list is ranked by combining several scorers. Following (Zhou et al., 2019), proposal and validation scores are applied. Additionally, a WSD model is used to get the WordNet definition (gloss) for both target and candidate given in context, and then similarity of two gloss embeddings is measured. The last scorer uses a fine-tuned STS (Semantic Textual Similarity) model (stsb-roberta-large) to compute the similarity between the original and candidate sentence.

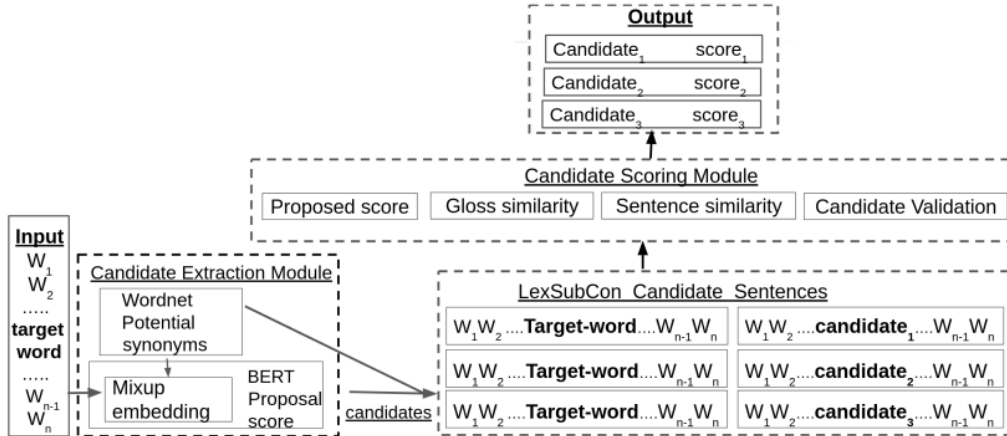


Figure 5 LexSubCon architecture.

2.3.6 ParaLS

ParaLS (Qiang et al., 2023) proposes using a paraphraser model to address two main limitations of previous approaches: (i) predictions are based on context rather than

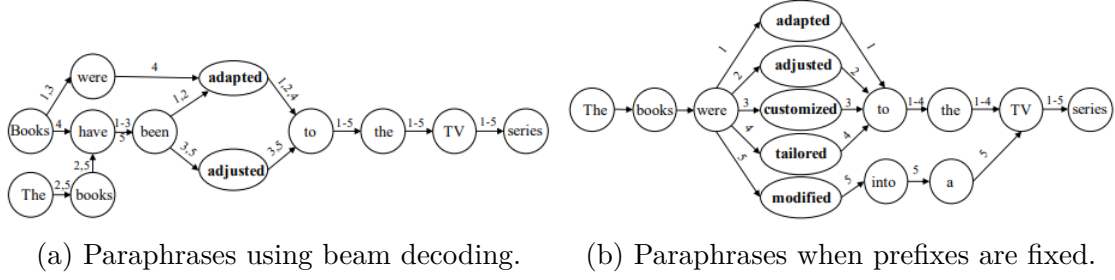


Figure 6 Paraphrase decoding strategies for the sentence "The books were adapted into a television series" and the target word "adapted".

meaning preservation focus; (ii) their inability to propose multi-token candidates. Neural paraphrasers use encoder-decoder architecture similar to translation models. To implement a paraphraser, the authors fine-tune the BART-base model using the paraphrase dataset ParaBank2. For candidate generation, the original sentence is fed into the model, the decoder is forced to begin with the same prefixes (words before target), and beam search is applied to obtain the next predictions. Another decoding strategy (ParaLS*) is also evaluated, which additionally uses a lookahead heuristic to adjust scores based on the probability to match the same suffixes. For candidate scoring, a linear combination of Paraphraser score, BARTScore (Yuan et al., 2021) and BLEURT (Sellam et al., 2020) is used. BARTScore and BLEURT both measure the likelihood of the candidate sentence given the original sentence. The results demonstrate state-of-the-art performance on LS task.

2.3.7 PromptSub

PromptSub (Shi et al., 2024) introduces the first decoder-only causal language model (CLM) for LS. Unlike the prior approaches that separate candidate generation and ranking, it employs a single-step method to produce candidates along with their scores. The authors fine-tune GPT-2 with prompts such as *The " w_x " in " S " can be substituted with " y ". </s>* where the underlined part is the prediction target. This prompt format grants full bidirectional context to a left-to-right CLM model. The prompt corpus is constructed from LST datasets using different substitute sampling strategies. Enriched prompt variants contain position, part-of-speech, lemma, multiple substitutes, and WordNet synsets. Beam search was used during inference.

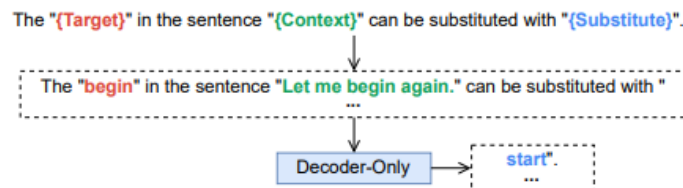


Figure 7 PromptSub inference.

3 Methodology and Implementation

3.1 Methodology Overview

Lexical substitution faces two main problems: recalling plausible candidates and scoring them with consistent accuracy. While transformer-based methods have made progress, important issues remain in both subtasks. For instance, MLM-based approaches can only propose single-token candidates, and embedding similarity often scores antonyms as highly similar.

In order to develop a high-quality solution, multiple approaches were experimented in both subtasks and their performance was systematically evaluated. A purely transformer-based solution was targeted, given the proven success of recent methods not relying on lexical resources. Initially, MLM-based approaches were reproduced such as (Zhou et al., 2019) as their findings were promising but the results reported by later studies were inconsistent, due to absence of a public implementation. In the second phase, MLM results were used as a baseline and CLMs were investigated, since they are largely unexplored in LS, and can produce multi-token substitutes.

Before starting programming right away, a third party implementation² for BERT-based method was found and examined for a couple of days. This period was valuable for developing essential skills, such as learning the details of BERT model, the LS methodology employed, and the technology stack, primarily PyTorch and the Transformers library. To ensure clarity, the codebase was refactored for readability, which also facilitated deeper comprehension. During this process, several issues were identified. For instance, dropout was being applied to the same embedding more than once, which was increasing the number of zeros in the embedding with each iteration. In addition, the contextual representation relied on the first four layers of BERT rather than the last four layers used in the original study. Further problems were revealed during debugging, but overall this initial exploration provided the necessary foundation to start a new implementation.

The development methodology followed established software engineering practices, including object-oriented design, clear naming conventions, and modular methods to enhance readability and productivity. Test Driven Development was employed, with unit tests ensuring stability, guiding the implementation of minimal working production code, and supporting safe refactoring throughout the cycle. For complex

² <https://github.com/jvladika/Lexical-Substitution>

computations relying on pretrained model outputs, intermediate values were reported at each calculation stage, enabling manual verification of correctness and consistency.

For effective analysis of individual substitution failures, development of a custom benchmark tool was preferred. Standard LS evaluation metrics and datasets were used to ensure fair comparison with prior studies. Hyperparameters were tuned only using the trial datasets. Significant benchmark snapshots were saved to track progress. Each report file name included the run date, model name, dataset, and custom changes. Randomness was eliminated by fixing seeds and setting temperature as zero. This determinism provided stable feedback during development, and ensured that rerunning benchmarks on earlier commits reproduced the original reports.

3.2 Implementation Overview

The project aims to identify the most effective transformer-based LS approaches for both candidate generation and scoring subtasks. To support this goal, the system architecture mainly consists of a benchmarking component called Benchmark Reporter and multiple substituters each implementing these subtasks. A command-line tool is used either to run substituters on a single input or to benchmark them.

Substituters implement the same pipeline to enable effective comparison across different approaches. Each component provides a method named *substitute*, which takes a Substitution Request object as an input and returns a Substitution Table object as output. Substitution Request is a simple data structure containing the text (context), target word, its position, optional Part-of-Speech (POS) tag, and an optional list of candidate words if they were already specified.

The output object, Substitution Table, has a dynamic structure. Each row corresponds to a candidate word, while the columns can contain different types of scores depending on the substituter implementation. It includes methods for conversion to and from a Pandas DataFrame. The table is ranked, with the top prediction in the first row followed by candidates with lower scores. Multiple scoring columns allow detailed observation of intermediate calculation steps and make error detection easier.

Substitution tables also include a standard boolean column indicating whether a candidate should be excluded from the prediction list. To ensure a fair comparison of different methods, the same exclusion rules are applied using an external Candidate Excluder. This component checks five conditions: the candidate must be a separate word (not a continuation token), must not have the same root with the target word, should have a compatible POS tag, must not be a stopword, and must not contain

```

def are_candidates_included(self, candidates, target="", tag=""):
    return [c.startswith(" ")
            and self.is_candidate_included(c.lower().strip(), target, tag)
            for c in candidates]

def is_candidate_included(self, candidate, target, tag=""):
    return (not self.has_same_root(candidate, target)
            and not any(p in candidate for p in string.punctuation)
            and not candidate in stopwords.words("english")
            and (not tag or tag in self.get_possible_tags(candidate)))

```

Listing 1 Candidate exclusion functions.

punctuation. Each of these filters was found to have a positive effect on benchmarks. Benchmark Reporter then considers only the top ten non-excluded candidates.

Although most substituters include strategies for both candidate generation and scoring, there are two exceptions: GPT-4 Substituter produces ranked candidates without explicit scores, and BART Substituter only scores candidates fetched from an external proposer. For candidate generation, all substituters first check the request itself, then use an external proposer if provided, and finally fall back to their own generation implementation. The consistent interface increases modularity, enabling hybrid approaches such as combining dropout-based proposals with LLaMa similarity scores.

The development environment comprised Python 3.11 on Linux. PyTorch, Hugging Face Transformers and llama-cpp-python libraries were used for model execution and inference. Git was employed for version control, PyCharm as the main IDE, and PyTest for unit testing. The system prefers CUDA GPUs but automatically falls back to CPU. Importantly, all substituters were able to run on a local GPU with 6 GB of VRAM. The pipeline loads models lazily and disables gradient computation during inference to reduce memory overhead. For more long-running experiments, execution was offloaded to Google Colab GPUs. A bash script (*setup_linux.sh*) was developed to automate project setup on Colab, allowing quick experimentation.

3.3 Console Tool

The project is run through a lightweight command-line interface that provides access to eight substitution methods. The tool supports three functions. The *substitute* command applies a chosen method to a single user input, returning the corresponding substitution table. The *demo* command runs a default example for a quick verification. The *benchmark* command executes a method on a dataset, displays progress while loading predictions, and produces a detailed report file. Secondary options such

```
Usage: word_replacer.py <Function> [Arguments]
Functions:
- demo <method>
- substitute <method> <text> <target> <position>
- benchmark <method> <dataset> [report_path]
Methods: zhou, dropout, concat, pattern, gpt2, gpt4, llama, bart
Datasets: lst_trial, lst_test, lst_all, coinco_trial, coinco_test, coinco_all
```

Listing 2 CLI tool usage.

as data shuffling, limiting samples, enabling GAP benchmarking mode, or adjusting hyperparameters are currently configured within the script.

3.4 Benchmark Reporter

The benchmarking process is organised into four stages. First, the dataset files are parsed and loaded into a single Pandas DataFrame, where each row represents one substitution request and includes a column for the gold map (human annotations). Second, the predictions from the chosen substituter are added as a new column, with each cell containing the top ten predictions for that instance. Third, evaluation scores are computed by comparing these predictions against the gold map; multiple scoring columns are appended to the table to capture different metrics. Finally, a benchmark report is generated, containing the full table, aggregate scores, and a list of skipped instances for later analysis.

```
def run_benchmark(substituter, dataset_name, report_path):
    reporter = BenchmarkReporter(Path(__file__).resolve().parents[0] / "
                                dataset", report_path, print_progress=True)
    reporter.load_dataset(dataset_name)
    reporter.load_predictions(substituter)
    reporter.load_scores()
    reporter.print_report()
```

Listing 3 Four benchmarking stages.

The LS07 (McCarthy & Navigli, 2007) and LS14 (Kremer et al., 2014) datasets share the same structure. Each trial and test subset consists of two files: one containing the sentences and another with gold substitution votes. The benchmarking component parses both files, matches the records, and loads them into a single table with columns for id, text, target, position, POS tag, and gold map. Non-ASCII bytes were cleared to prevent display issues such as missing characters. Following prior studies (Michalopoulos et al., 2021), multi-word gold substitutions and records without gold data were also removed.

In the next stage, predictions are retrieved from the selected substituter for each row. Substituter-specific scores are ignored, as the evaluation metrics require only a ranked list of candidates. Earlier substituter implementations used token indices for positions, but the benchmark datasets rely on word indices, therefore the substituters were updated for compatibility. To ensure consistency, each substituter checks whether the target word matches the lemmatized form of the word at the specified position. If this validation fails, or any other error occurs, the instance is flagged in the report and excluded from the final score calculation.

Evaluation scores are computed by comparing the gold map against the prediction list. Because the gold substitutions are given in their base forms, predictions are lemmatized before comparison. The implemented metrics are briefly described below.

- **Best Score:** The percentage of gold substitution votes assigned to the top-ranked (best) prediction.
- **Best Mode Score:** A binary measure that is 1 if the best prediction matches the highest-voted gold substitute.
- **OOT Score:** *Out of Ten* score sums the gold vote percentages of all predictions in the top-ten list.
- **OOT Mode Score:** A binary measure that is 1 if any of the top ten predictions match the highest-voted gold substitute.
- **Precision@1:** Indicates whether the top-ranked prediction appears among the gold substitutes.
- **Precision@3:** The proportion of the top three predictions that appear among the gold substitutes.
- **Recall@10:** The proportion of gold substitutes covered within the top-ten predictions.
- **GAP Score:** Generalised Average Precision evaluates how well predictions are ranked relative to the vote weights of gold substitutes.

Once all scores are added to the table, final averages are calculated. Following standard practice in LS evaluation, Best Mode and OOT Mode scores are excluded from the average when multiple gold substitutes have the same highest vote.

Only for GAP score, an external implementation³ was used to ensure consistency with other studies. However, despite using the same calculation, significant discrep-

³ <https://tinyurl.com/gap-measure>

ancies were observed. To investigate, several studies (Arefyev et al., 2022; Melamud et al., 2016; Michalopoulos et al., 2021; Qiang et al., 2023) were attempted to reproduce using their public implementations, though most efforts were unsuccessful due to dependency conflicts and outdated requirements. The LexSubCon project was successfully built, and its GAP outputs revealed the key difference: candidates were supplied to substituters from the datasets rather than generated by the models. Once the same approach was adopted, the GAP scores aligned with those reported in the literature. This was implemented by adding a *provide_candidates* option. When this flag is enabled, *BenchmarkReporter* collects substitutions for the target word from multiple instances and includes these candidates in the *SubstitutionRequest*.

Column	Value
id	859
target	check
position	2
text	i have checked multiple times with my order and that is not the case .
tag	v
gold_map	{'verify': 4, 'confirm': 1, 'investigate': 1}
tie	False
predictions	[verified, looked, inspected, searched, locked, adjusted, scanned, cleared, read, updated]
best_score	0.666667
best_mode_score	1.0
oot_score	0.666667
oot_mode_score	1.0
precision@1	1.0
precision@3	0.333333
recall@10	0.333333
gap_score	0.470588

Table 1 Example row in a benchmark table.

3.5 Substituters

This section introduces the substituter implementations developed in the project. Several MLM-based approaches are implemented within the *BertSubstituter* class, with the *PatternSubstituter* providing additional preprocessing functionality. CLM-based methods are represented by the *Gpt2Substituter*, *Gpt4Substituter*, and *LlamaSubstituter*, while the *BartSubstituter* extends existing outputs by appending BART-based scores. Together, these components enable evaluation of a wide range of strategies. A high-level overview of substituter instances is presented below.


```

if name == "zhou":
    return BertSubstituter("bert-large-uncased", iteration_count=5,
                           score_basis="validation_score")
if name == "dropout":
    return BertSubstituter("roberta-base", dropout_rate=0.3,
                           iteration_count=5, alpha=0.01)
if name == "concat":
    return BertSubstituter("roberta-base", concatenate=True,
                           dropout_rate=1, use_mask_token=True)
if name == "pattern":
    return PatternSubstituter(get_substituter("dropout"),
                              ["%", "or", "%"], position_change=2)
if name == "gpt2":
    return Gpt2Substituter("gpt2-large", pll_enabled=True)
if name == "gpt4":
    return Gpt4Substituter("gpt-4o", temperature=0.7)
if name == "llama":
    return LlamaSubstituter(target_similarity_enabled=True,
                           sentence_similarity_enabled=True)
if name == "bart":
    return BartSubstituter(proposer=get_substituter("llama"))

```

Listing 4 Overview of substituter instances.

3.5.1 BERT Substituters

The *BertSubstituter* class was the first implementation of the project, following the BERT-based approach (Zhou et al., 2019) described in Subsection 2.3.1. While it can be configured to match the original study conditions, this component extends this approach with a wider range of options. For example, it supports both BERT and RoBERTa model families. In addition to the original dropout method, it enables alternative target word injection techniques such as Duplicate Input and Dynamic Patterns. The hyperparameters from the original study were also made configurable.

```

class BertSubstituter:
    def __init__(self,
                  model_name,
                  dropout_rate = 0.3,
                  candidate_count = 50,
                  alpha = 0.01,
                  iteration_count=1,
                  deterministic=True,
                  concatenate=False,
                  use_mask_token=False,
                  score_basis="target_similarity"):
        ...

```

Listing 5 BertSubstituter constructor parameters with default values.

The substitution process begins with the input text tokenization using the tokenizer of the selected model. The *position* parameter is converted to target token index, by tokenizing the text up to the word in position, then taking the length of the resulting

array. This process changes slightly when concatenation (Duplicate Input) is enabled. The identified target token is then checked whether it has the same root with the target word provided in the substitution request. This validation may fail for rare target words due to tokenization issues.

To get the model predictions, the token ids are converted to input embeddings and fed into the model with the partially masked target embedding. The probabilities for vocabulary tokens are obtained by applying softmax to output logits in the target position. This process is repeated for several iterations to reduce the randomness introduced by dropout. Earlier versions repeated all scoring phases for iterations, but this was later optimised to iterate only the non-deterministic part. If *deterministic* flag is enabled, the iteration index is used as the seed for generating random dropout indices, making the experiment results reproducible.

```

vocab_probs = torch.zeros(self.get_vocabulary_size(), dtype=torch.float32)
for i in range(self.iteration_count):
    masked_embeddings = self.mask_target(clear_embeddings, target_index,
                                         self.dropout_rate, i)
    model_output = self.get_output_from_embeddings(masked_embeddings)
    vocab_probs += self.get_vocab_probs(model_output, 0, target_index)
return vocab_probs / self.iteration_count

```

Listing 6 Average vocabulary token probabilities using iterations.

The top entries in this probability distribution are selected as candidates. The proposal scores are calculated based on the raw probabilities. For analysis and correctness, the validation score logic is divided into two phases: the similarity matrix and the attention vector. Multiplication of these produces validation scores.

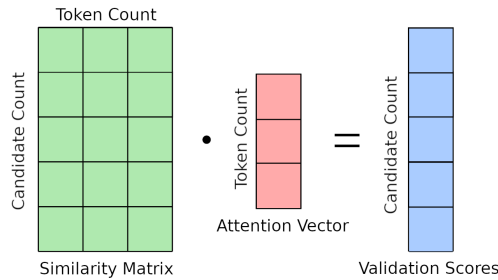


Figure 8 Validation score calculation.

For similarity matrix, initially the candidate sentences are generated, with each sentence containing a different candidate word in its target position. The encodings of these sentences are then fed into the model as a batch input for efficiency. The model output for this batch input is compared to the output of the original input to produce the similarity matrix, where each row corresponds to a candidate sentence, each col-

umn represents a token position, and each cell indicates the similarity of contextual representations between the original and the candidate token.

	<s>	The	wine	is	too	strong	to	drink	.	</s>
... strong ...	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
... weak ...	0.9818	0.9947	0.9910	0.9873	0.9740	0.8769	0.9570	0.9762	0.9933	0.9939
... hot ...	0.9735	0.9928	0.9898	0.9799	0.9549	0.7636	0.9764	0.9628	0.9798	0.9885
... stronger ...	0.9818	0.9973	0.9967	0.9874	0.9834	0.9218	0.9745	0.9862	0.9932	0.9929
... powerful ...	0.9778	0.9964	0.9947	0.9806	0.9775	0.9159	0.9827	0.9827	0.9909	0.9885

Figure 9 Similarity Matrix: token similarities between original-candidate sentences.

The attention vector is obtained by averaging attention matrices across all heads and layers, following (Zhou et al., 2019). Initially, the k th row of the averaged matrix (with k as the target index) was used, but this gave inflated validation scores due to the attention dominance of special tokens. The attention direction was corrected by instead taking the k th column, which gives attentions from other tokens to the target token. This column vector is then normalised to sum to 1.

In addition to the validation score, the substitution table also includes two alternative similarity measures: target similarity (the k th column of the similarity matrix) and [CLS] token similarity (the first column for BERT models). The *score_basis* parameter allows choosing any of these as the main similarity metric.

Based on this substituter, alternative target word injection techniques were implemented. For the Duplicate Input technique, *concatenate* and *use_mask_token* options are enabled, with the *dropout_rate* set to 1 for full masking. For Dynamic Patterns, an external preprocessing component replaces the target word with the specified pattern before passing it to the underlying substituter.

candidate	is_included	candidate_prob	normalized_prob	prop_score	cls_sim	target_sim	val score	final score
run	False	0.726626	2.657998	0.977573	1.000000	1.000000	1.000000	1.009776
ran	False	0.039499	0.144489	-1.934553	0.996901	0.974509	0.994016	0.974670
appeared	True	0.082916	0.303307	-1.193011	0.989470	0.902508	0.981106	0.969176
published	True	0.116600	0.426524	-0.852087	0.984861	0.892741	0.964221	0.955700
featured	True	0.004554	0.016659	-4.094789	0.987060	0.874488	0.965941	0.924993
been	False	0.005875	0.021490	-3.840179	0.985590	0.897265	0.962256	0.923854
written	True	0.011324	0.041424	-3.183897	0.976830	0.853999	0.954035	0.922196
printed	True	0.005101	0.018659	-3.981418	0.973085	0.851237	0.950605	0.910791

Figure 10 Substitution table produced by a BERT-based substituter for the input sentence "chapters from this book by paul collins have run in mcsweeney 's quarterly concern." and target word "run".

3.5.2 GPT Substituters

This subsection describes the initial CLM-based substituters, implemented using the GPT-2 models and a GPT-4o client in two separate classes. *Gpt2Substituter* follows a similar approach to *BertSubstituter* for candidate generation, then ranks candidates

using the next-token probabilities and sentence log-likelihood scores. In contrast, *Gpt4Substituter* does not include a scoring mechanism and instead retrieves the candidate list from the OpenAI API via a prompt.

For candidate generation, *Gpt2Substituter* employs beam search to extract the top fifty most probable tokens. Unlike BERT-based models, GPT models lack access to right-hand context, so the full sentence is provided in the input prompt. Initially, prompts from Subsection 2.3.7, following (Shi et al., 2024), were used. However, this format caused word tokens (prefixed with a space) to receive disproportionately low probabilities due to the quotation mark at the end of input. After several tests, the prompt was revised to a structure similar to Duplicate Input, where the original sentence is concatenated with the truncated sentence up to the target position.

For scoring, sentence log-likelihood is used alongside raw token probabilities. This measure captures how fluent or natural the candidate sentence is, rather than its semantic similarity to the original sentence. This scoring was chosen in order to take the sentence continuation into account, which was overlooked for some next-token predictions. The likelihood is computed by summing the log probabilities of all input tokens, where each token’s probability is conditioned on its preceding context. For each output position in the model, only the log probability of the next (true) token is retrieved. The sum of these probabilities is then divided by the token count, to ensure compatibility across sentences with different lengths. For integration, likelihood scores and raw probabilities are converted into integer ranks, the two ranks are then summed for final ranking.

candidate	is_included	probability	pll_score
rough	False	0.966800	-3.645744
tough	True	0.011467	-3.825946
rou	False	0.004419	-4.215533
.	False	0.002151	-4.250764
very	False	0.002135	-4.229788
difficult	True	0.001043	-4.012004
bad	True	0.000998	-3.943766
hard	True	0.000877	-3.958331
rocky	True	0.000439	-3.998443
poor	True	0.000426	-3.980694
pretty	True	0.000360	-4.345698
good	True	0.000304	-4.105964
really	False	0.000291	-4.341017

Figure 11 Substitution table produced with GPT-2 model for the input sentence "col. berry led again so it was a rough job ." and target word "rough".

Gpt4Substituter was developed mainly for comparison with other approaches. It is the only component in the project that relies on a remote API rather than a local model. A simple prompt is used to obtain a JSON-formatted list of ten single-word substitutes from the GPT-4o model. While the substitutes are not scored, they can

still be evaluated within the benchmarking framework, which only requires a ranked candidate list. When candidates are already provided in the substitution request, an alternative prompt is used to sort these existing candidates.

3.5.3 LLaMa Substituter

Following the work with GPT-2 and GPT-4o, LLaMa was considered as a large and open alternative, offering the opportunity to conduct LLM experiments locally. The Llama-3.1-8B-Instruct-Q4_K_M variant was chosen as it provides a good trade-off between precision and VRAM requirements. The model is instruction-tuned, and quantized to reduce GPU memory usage.

After obtaining the model from the Hugging Face repository, several integration options were evaluated. The Ollama library, which relies on a local REST API, was not selected since it limits access to model internals. An initial attempt combined the Transformers library with BitsAndBytes for quantization, but this resulted in GPU out-of-memory errors. As an alternative, the model was downloaded in GGUF format and first tested via the llama.cpp CLI tool. It was then integrated using the llama-cpp-python library, with stability achieved by adjusting the library version and configuring the model's context size.

```
self.model = llama_cpp.Llama.from_pretrained(
    repo_id="bartowski/Meta-Llama-3.1-8B-Instruct-GGUF",
    filename="Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf",
    n_ctx=2048,
    n_gpu_layers=-1,
    seed=0,
    chat_format="llama-3",
    verbose=False,
    embedding=True)
```

Listing 7 Llama model configuration.

For candidate generation, the substituter uses a prompt rather than beam search in order to include multi-token words. Temperature was set to zero to ensure deterministic and probable results. The prompt was optimized using the LS07 trial dataset. A minimal prompt asking ten replacements was used, since detailing it with requirements did not always produce compliant outputs. To increase model recall, an alternative prompt requested a contextual explanation first, but this approach introduced side-effects such as excluding words that were mentioned in the explanation. Instead of a JSON response, a plain list was requested, to prevent low probabilities for tokens starting with space when the last character is a quotation mark. A post-processing step is applied, which parses the list, prevents multi-word replacements by

keeping only the last word from each item, and removes duplicate entries. Candidate exclusion was disabled, as it was found to reduce performance for LLM outputs.

For candidate ranking, two main scoring strategies were implemented: target similarity and sentence similarity. For both measures, initially the candidate sentences are generated, each containing a different candidate word in its target position. Target similarity compares the contextual embeddings at the target positions of the original and candidate sentences. It uses only the last-layer representations, in contrast to last four layers used in *BertSubstituter*.

A key challenge in scoring is the varying number of tokens across candidates. For example, the original word "strong" consists of one token, while the candidate "pungent" has three tokens and "overpowering" has two tokens according to the LLaMa tokenizer. Since embedding similarity requires vectors of the same size, multiple token embeddings must be reduced to a single vector representing the word. The initial approach applied mean pooling, which averages values across embeddings. Later, keeping only the last token embedding was found to be more effective. Therefore, target similarity uses only the last token embeddings of the candidate and original words. Similarly, for sentence similarity, sentence embeddings are obtained by concatenating token embeddings in the sentence, but excluding the target word tokens except its last token. This method ensures that both target and sentence embeddings are fixed in size.

```
def get_embeddings_with_one_from_position(self, text, position):
    embeddings = self.get_token_embeddings(text)
    first_token_index = self.get_first_token_index(text, position)
    last_token_index = self.get_last_token_index(text, position)
    return embeddings[0:first_token_index] + embeddings[last_token_index:]
```

Listing 8 Sentence token embeddings with only the last token from target position.

After obtaining fixed-size candidate embeddings, both scores follow the same procedure. The dot product is used to calculate similarity, and the resulting score is divided by the score for the original word (its similarity with itself), producing normalized candidate scores between 0 and 1. Both measures were implemented as optional, allowing faster benchmarking when only one score or the proposal ranking is required.

Finally, BARTScore was included in the project as an external implementation ⁴, motivated by its strong performance reported in (Qiang et al., 2023). It was intended to serve as a complementary likelihood measure alongside LLaMa similarity scores.

⁴ https://github.com/neulab/BARTScore/blob/main/bart_score.py

candidate	index	is_included	target_sim	sentence_sim
fear	0	False	1.000000	1.000000
dread	2	True	0.769445	0.917313
worry	8	True	0.818180	0.907497
panic	5	True	0.739782	0.880298
dismay	9	True	0.520303	0.876693
concern	7	True	0.604483	0.869008
apprehension	3	True	0.424456	0.854884
anxiety	1	True	0.609992	0.851328
terror	4	True	0.425023	0.846038
alarm	6	True	0.375126	0.828304

Figure 12 Substitution table produced with LLaMa model for the input sentence "nothing, I fear, I haven't said already." and target word "fear".

To achieve this, a dedicated *BartSubstituter* class was developed, which accepts any substituter in its constructor and appends BART scores as an additional column in the substitution table.

4 Analysis and Evaluation

This chapter evaluates a broad set of lexical substitution approaches through systematic benchmarking. In addition to comparing the default substituters, controlled experiments were carried out to examine the impact of specific parameters and techniques. Although trial datasets were used during development for tuning and debugging, all results reported here are based on the test datasets. All experiments were executed on Google Colab using an NVIDIA A100 GPU to ensure consistent runtime conditions. Overall, the RoBERTa base model combined with dropout and target similarity outperformed other MLM-based methods, while larger CLMs (except GPT-2) tended to achieve superior results. The following sections present these findings in detail, each with a table focusing on a different aspect.

4.1 Dropout Implementations

Initially, the findings of (Zhou et al., 2019) were compared with the project’s reimplementation using the same parameters. Since several studies (Arefyev et al., 2022; Lacerra et al., 2021; Michalopoulos et al., 2021) reported difficulties in reproducing the original results, providing an open implementation with comparable performance was one of the project’s aims. Although the original scores could not be fully produced, the results achieved are closer than those of the (Michalopoulos et al., 2021) reimplementation.

LS07							
Implementation	best	best-m	oot	oot-m	P@1	P@3	R@10
Original Imp.	20.3	34.2	55.4	68.4	51.1	-	-
LexSubCon Imp.	12.8	22.1	43.9	59.7	31.7	-	-
Project Imp.	18.8	29.5	54.7	65.0	43.3	30.7	45.3

Table 2 Implementations of the same method. Original findings (Zhou et al., 2019), reimplementation by (Michalopoulos et al., 2021), and the project reimplementation.

4.2 Similarity Measures

This section compares three candidate scoring measures among MLM methods. The implementation with the original study conditions was used as a baseline, and the validation score was replaced with two alternative measures by adjusting the *score_basis* parameter in *BertSubstituter*, while keeping all other parameters fixed. Following the original study, *bert-large-uncased* model was used in these measurements.

Interestingly, target word similarity was found to outperform the originally used validation score, which considers similarities of not only the target words, but also the surrounding words. The same trend was observed during development with the trial dataset, leading deeper analysis and debugging of the validation score logic, since the original study findings indicated validation score as superior to target similarity. The validation score calculation was separated into two components, the token similarity matrix and the average attention weights, and both were found to behave consistently. However, the contextual representations of the surrounding tokens between original and candidate sentences were generally highly similar, which appeared to affect the validation score negatively. Target similarities, by contrast, were not influenced by this effect and therefore provided a more varying measure.

Additionally, [CLS] token similarity was experimented, as this token is designed to capture the sentence meaning. While the results are comparable, they consistently underperformed compared to other measures. Finally, different numbers of layers were examined for contextual representation, with four layers found to be optimal, consistent with the original study.

LS07							
Measure	best	best-m	oot	oot-m	P@1	P@3	R@10
Validation Score	18.8	29.5	54.7	65.0	43.3	30.7	45.3
CLS Similarity	17.7	27.8	54.2	65.0	41.2	29.6	44.9
Target Similarity	21.6	35.0	60.5	68.6	49.5	36.7	49.3

Table 3 Comparison of similarity measures. All other parameters are fixed. The Validation Score row corresponds to the Project Implementation row in Table 2.

4.3 Masked Language Models

This section evaluates the impact of different masked language models on substitution performance. During trial experiments, *roberta-base* consistently outperformed *bert-large-uncased*, and the test benchmarks confirm this trend, although the margin is smaller on the test dataset. All models here were evaluated using the target similarity measure with identical parameter settings. The results show *roberta-base* achieving the strongest overall performance, whereas *roberta-large* unexpectedly lags behind both BERT variants. Closer inspection of substitution tables revealed that BERT models are more prone to assigning high probabilities to punctuation and word continuation tokens, particularly when input sentences lack closing punctuation. Applying exclusion filters for such tokens improved BERT performance, narrowing the gap with RoBERTa.

LS07							
Model	best	best-m	oot	oot-m	P@1	P@3	R@10
bert-base-uncased	21.3	34.5	55.9	64.7	48.5	33.3	45.2
bert-large-uncased	21.6	35.0	60.5	68.6	49.5	36.7	49.3
roberta-base	22.2	36.1	57.5	70.6	50.9	35.4	49.9
roberta-large	19.9	32.0	55.1	70.8	46.4	32.7	48.9

Table 4 Comparison of masked language models. All other parameters are fixed. The bert-base-uncased row corresponds to the Target Similarity row in Table 3.

4.4 Target Injection Techniques

Several alternatives to the embedding dropout technique were explored. Initially, the effect of dropout rate was observed, where low rates produced synonyms close to target word but overlooked context, whereas high rates yielded words fitting in context but unrelated to the original meaning. The rate of 0.3, used in the original study, was found optimal. The Duplicate Input technique achieved slightly lower scores but runs faster, as it avoids multiple iterations and produces deterministic results in a single pass.

For Dynamic Patterns, multiple text modifications were tested, such as appending parentheses or adding the word "or"/"and" followed by a mask. The final implementation used the pattern "T or <MASK>", where T is the target word. These approaches generally hurt performance as they alter the sentence structure, but combining the pattern with partial masking (dropout) improved results, though still below standard dropout. For approaches using a full mask (Duplicate Input and Pattern + Full Mask), reducing the proposal score weight using the α parameter improved performance, suggesting that full masking makes raw predictions less reliable.

LS07							
Technique	best	best-m	oot	oot-m	P@1	P@3	R@10
Embedding Dropout	22.2	36.1	57.5	70.6	50.9	35.4	49.9
Duplicate Input	21.3	35.7	56.2	70.5	47.0	33.2	49.0
Pattern + Full Mask	17.1	27.2	47.7	59.9	40.1	28.6	40.5
Pattern + Dropout	21.7	34.7	56.2	70.8	49.2	34.3	49.4

Table 5 Comparison of target injection techniques. All other parameters are fixed. The Embedding Dropout row corresponds to the roberta-base row in Table 4.

4.5 Exclusion Filters

Table 6 shows the effect of exclusion filters on the best predictions of BERT and GPT-2. Both models often ranked words sharing the same root as the target due to

target word injection, and applying a root-based filter reduced this issue. Additional filters for punctuation, stopwords, and POS tags further improved performance, with the exception of a small decrease for BERT when stopwords filtering was applied.

LS07		
Filter	BERT	GPT-2
Separate Word Token	0.2	0.1
+ Not the Target Word	14.7	9.1
+ Unrelated to Target	20.7	12.0
+ Without Punctuation	20.8	14.3
+ Not a Stopword	20.6	15.6
+ Compatible POS Tag	21.6	17.8

Table 6 Comparison of "best" scores when additional filters applied.

4.6 CLM Methods

Several CLM models were tested using different approaches. GPT-4o and LLaMa generated candidates via prompting, while GPT-2 relied on beam search, as direct generation did not produce reliable outputs. GPT-2 applied the standard candidate exclusion filter, but this was disabled for GPT-4o and LLaMa, where it reduced performance. Overall, compared to GPT-4o, LLaMa candidates were less accurate, more repetitive, and more influenced by preceding outputs, which is an expected outcome given the smaller size of LLaMa (8B) relative to GPT-4o (200B).

GPT-2 beam search performed worse than MLM-based methods, since the same sentence continuation could not be enforced during prediction, even when the model was given access to full sentence by duplicating the input. This limitation was mitigated by integrating a sentence likelihood measure, where each candidate was assigned an additional score based on how well it fits into the right-hand context. This step improved the performance to the level of the Duplicate Input technique.

GPT-4o candidates obtained via prompt outperformed all other methods tested in the project. Although this approach does not include a candidate scoring mechanism, it provides a useful reference for how an ideal substituter with deterministic scoring might perform. Two temperature configurations were tested, but these did not significantly affect performance.

For LLaMa, three setups were evaluated: the first used the raw model response as ranking, the second re-ranked candidates by target similarity, and the third by sentence similarity. Among these, the raw model response achieved the best results.

However, sentence similarity provided an effective candidate scoring measure, which will be examined further in the following section.

LS07							
Method	best	best-m	oot	oot-m	P@1	P@3	R@10
GPT-2 Beam	17.8	28.9	51.1	66.9	41.6	29.9	44.8
GPT-2 Beam + Likelihood	21.3	34.7	53.8	69.2	49.0	33.4	47.0
GPT-4o Proposals (T=0)	30.6	49.4	71.6	83.7	68.8	48.7	67.1
GPT-4o Proposals (T=0.7)	30.1	48.3	71.2	83.3	68.6	47.6	66.6
LLaMa Proposals (T=0)	24.9	40.3	59.5	72.4	57.8	39.7	54.6
LLaMa + Target Sim.	20.7	33.3	59.3	72.2	49.5	37.4	54.4
LLaMa + Sentence Sim.	22.4	35.0	59.3	72.2	53.4	38.4	54.4

Table 7 Comparison of different CLM methods applied.

4.7 Candidate Ranking

The final experiment evaluates different candidate scoring methods by their ranking quality using the GAP (Generalised Average Precision) measure. For this task, all scoring methods were provided with the same candidates, which are obtained from all gold substitutions for that target word across multiple instances in the dataset.

Dropout Proposal+Target Sim.		Llama Sentence Similarity	
Candidate	Score	Candidate	Score
exam	0.949151	consultation	0.974101
assessment	0.932262	investigation	0.968353
analysis	0.926458	study	0.965593
inspection	0.924426	assessment	0.965229
test	0.907851	inspection	0.958443
investigation	0.900694	test	0.957316
scrutiny	0.887252	analysis	0.947974
study	0.885656	exam	0.941249
consultation	0.877616	scrutiny	0.939915
enquiry	0.763735	enquiry	0.928415

Figure 13 Side by side candidate ranking comparison for RoBERTa Dropout and LLaMa sentence similarity measures, with the same set of candidates. The input sentence is "From the previous examination last week the auto-immune deficiency could also be excluded." and the target word is "examination". Annotator preferences are "investigation" (2), "inspection" (2), "consultation"(1), "assessment" (1).

Despite employing a standard external GAP implementation and consistent methodology, small discrepancies remained compared with the findings of earlier studies (Michalopoulos et al., 2021; Zhou et al., 2019). The cause of this difference could not be identified, though scores at the individual sample level were consistent.

Among the evaluated methods, LLaMa sentence similarity outperformed all others by a notable margin. This approach computes a simple dot product between concatenated output embeddings of the original and candidate sentences. The next best results were achieved by combining proposal and target similarity scores using the RoBERTa model with the dropout technique.

LS07	
Scoring Method	GAP
BERT Dropout s_p, s_v	50.9
RoBERTa Dropout s_p, s_t	54.8
RoBERTa Duplicate Input s_p, s_t	51.1
RoBERTa Dynamic Pattern s_p, s_t	53.0
GPT-2 Beam	47.9
BART Score	52.6
LLaMa Sentence Similarity	61.1

Table 8 Comparison of scoring measures for the candidate ranking task. s_p : Proposal Score, s_v : Validation Score, s_t : Target Similarity Score.

5 Conclusions and Future Work

This thesis investigated several ways to enhance lexical substitution quality. A key objective was to develop a system capable of producing a broad range of replacement suggestions accompanied by highly accurate scores. To achieve this, multiple strategies were implemented and their performance was measured to track progress. First, existing MLM methods were reimplemented and extended with several modifications. Next, a higher-quality CLM-based solution was pursued, given the comparatively limited attention these models have received in LS research. In both architectures, some established and novel techniques were applied. Overall, although CLM approaches improved both candidate generation and scoring, notable limitations remain.

For candidate generation, MLM and CLM approaches exhibit different limitations. The main disadvantage of MLM proposals is their inability to produce multi-token suggestions, which excludes many words with various suffixes and prefixes, as well as less common words in root form. Although CLM candidates obtained through prompting solves this issue, it introduces other problems such as repeated candidates, limited recall of suitable alternatives, inaccurate candidates influenced by prior predictions, and a significantly slower generation process. Experiments with GPT-4o indicated that some of these issues can be mitigated as model size increases. However, a more efficient solution might be applying beam search by concatenating each prediction with its next most probable tokens to allow multi-token replacements, and repeating this process over several iterations to expand the resulting word list. Due to time constraints, this CLM approach was left for future work.

The candidate scoring phase can also be improved in several ways. Although LLaMa sentence similarity outperformed other measures in the candidate ranking task, closer analysis revealed that it does not always fully capture the semantic differences between two sentences. A common drawback of embedding similarity measures is their tendency to assign high scores to antonyms or semantically distant words that share a common theme. One potential improvement to address this issue could involve averaging the similarities between a single candidate and multiple precise substitutions obtained from an LLM. Additionally, an ensemble score might balance different aspects, such as sentence fluency via GPT-2 or BARTScore likelihood, combined with semantic similarity from contextual and input embedding similarities. Such an ensemble could yield more robust results than relying on individual measures, which often fail in specific cases.

Other limitations in this study were related to evaluation quality and efficiency. To allow experimentation with more approaches, some operations such as LLaMA similarity and GPT-2 likelihood scoring were not fully optimised. For instance, in both cases, contextual embeddings could have been obtained more efficiently by processing candidate sentences as a batch input to the model. This optimisation significantly accelerated benchmarking in *BertSubstituter* when using a GPU. In contrast, CLM approaches generally required more time to evaluate. Consequently, larger and more diverse datasets such as CoInCo (Kremer et al., 2014) could not be fully utilised. Furthermore, the SWORDS (Lee et al., 2021) dataset may provide a more precise evaluation, as it does not rely on human recall for substitutions.

This project demonstrated the potential and limitations of transformer-based LS approaches through reimplementations, extensions, and evaluations of both MLM and CLM models. By refining existing methods and exploring CLM strategies, it provided valuable insights into how model architecture, size, scoring methods, and additional techniques affect substitution quality. The work highlighted challenges such as multi-token candidates and balancing semantic accuracy with sentence fluency, while offering directions for improvement through optimised candidate generation and ensemble scoring. The project also delivers an open framework enabling reproducible experiments and future development. These contributions provide a foundation for future research in lexical substitution.

Bibliography

- Arefyev, N., Sheludko, B., Podolskiy, A., & Panchenko, A. (2020, December). Always keep your target in mind: Studying semantics and improving performance of neural lexical substitution. In D. Scott, N. Bel, & C. Zong (Eds.), *Proceedings of the 28th international conference on computational linguistics* (pp. 1242–1255). International Committee on Computational Linguistics. <https://doi.org/10.18653/v1/2020.coling-main.107>
- Arefyev, N., Sheludko, B., Podolskiy, A., & Panchenko, A. (2022). Always keep your target in mind: Studying semantics and improving performance of neural lexical substitution. *arXiv preprint arXiv:2206.11815*.
- Biemann, C. (2013). Creating a system for lexical substitutions from scratch using crowdsourcing. *Language Resources and Evaluation*, 47(1), 97–122.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 4171–4186.
- Dinu, G., & Lapata, M. (2010). Measuring distributional similarity in context. *Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing*, 1162–1172.
- Erk, K., & Padó, S. (2008). A structured vector space model for word meaning in context. *Proceedings of the 2008 conference on empirical methods in natural language processing*, 897–906.
- Fellbaum, C. (1998). *Wordnet: An electronic lexical database*. MIT press.
- Hassan, S., Csomai, A., Banea, C., Sinha, R., & Mihalcea, R. (2007). Unt: Subfinder: Combining knowledge sources for automatic lexical substitution. *Proceedings of the fourth international workshop on semantic evaluations (SemEval-2007)*, 410–413.
- Hintz, G., & Biemann, C. (2016). Language transfer learning for supervised lexical substitution. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 118–129.
- Kremer, G., Erk, K., Padó, S., & Thater, S. (2014). What substitutes tell us-analysis of an “all-words” lexical substitution corpus. *Proceedings of the 14th Conference of the European Chapter of the Association for Computational Linguistics*, 540–549.

- Lacerra, C., Tripodi, R., Navigli, R., et al. (2021). Genesis: A generative approach to substitutes in context. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.
- Lee, M., Donahue, C., Jia, R., Iyabor, A., & Liang, P. (2021). Swords: A benchmark for lexical substitution with improved data coverage and quality. *arXiv preprint arXiv:2106.04102*.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). Roberta: A robustly optimized bert pre-training approach. *arXiv preprint arXiv:1907.11692*.
- McCarthy, D., & Navigli, R. (2007). Semeval-2007 task 10: English lexical substitution task. *Proceedings of the fourth international workshop on semantic evaluations (SemEval-2007)*, 48–53.
- Melamud, O., Goldberger, J., & Dagan, I. (2016). Context2vec: Learning generic context embedding with bidirectional lstm. *Proceedings of the 20th SIGNLL conference on computational natural language learning*, 51–61.
- Melamud, O., Levy, O., & Dagan, I. (2015). A simple word embedding model for lexical substitution. *Proceedings of the 1st workshop on vector space modeling for natural language processing*, 1–7.
- Michalopoulos, G., McKillop, I., Wong, A., & Chen, H. (2021). Lexsubcon: Integrating knowledge from lexical resources into contextual embeddings for lexical substitution. *arXiv preprint arXiv:2107.05132*.
- Mihalcea, R., & Faruque, E. (2004). Senselearner: Minimally supervised word sense disambiguation for all words in open text. *Proceedings of SENSEVAL-3, the Third International Workshop on the Evaluation of Systems for the Semantic Analysis of Text*, 155–158.
- Qiang, J., Li, Y., Zhu, Y., Yuan, Y., & Wu, X. (2020). Lexical simplification with pretrained encoders. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05), 8649–8656.
- Qiang, J., Liu, K., Li, Y., Yuan, Y., & Zhu, Y. (2023). Parals: Lexical substitution via pretrained paraphraser. *arXiv preprint arXiv:2305.08146*.
- Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. (2018). Improving language understanding by generative pre-training.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8), 9.
- Sellam, T., Das, D., & Parikh, A. P. (2020). Bleurt: Learning robust metrics for text generation. *arXiv preprint arXiv:2004.04696*.
- Shi, N., Hauer, B., & Kondrak, G. (2024). Lexical substitution as causal language modeling. *Proceedings of the 13th Joint Conference on Lexical and Computational Semantics (*SEM 2024)*, 120–132.

- Szarvas, G., Biemann, C., & Gurevych, I. (2013). Supervised all-words lexical substitution using delexicalized features. *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1131–1141.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Wu, Q., Burges, C. J., Svore, K. M., & Gao, J. (2010). Adapting boosting for information retrieval measures. *Information Retrieval*, 13(3), 254–270.
- Yuan, W., Neubig, G., & Liu, P. (2021). Bartscore: Evaluating generated text as text generation. *Advances in neural information processing systems*, 34, 27263–27277.
- Yuret, D. (2007). Ku: Word sense disambiguation by substitution. *Proceedings of the Fourth International Workshop on Semantic Evaluations (SemEval-2007)*, 207–214.
- Yuret, D. (2012). Fastsubs: An efficient and exact procedure for finding the most likely lexical substitutes based on an n-gram language model. *IEEE Signal Processing Letters*, 19(11), 725–728.
- Zhou, W., Ge, T., Xu, K., Wei, F., & Zhou, M. (2019). Bert-based lexical substitution. *Proceedings of the 57th annual meeting of the association for computational linguistics*, 3368–3373.

A Appendix: Sample Benchmark Report

Skipping: see != saw at 5
 Skipping: fall != fell at 15
 Skipping: fall != fell at 31
 Skipping: fall != fell at 5
 Skipping: fall != fell at 39
 Skipping: fall != fell at 18

 Final scores:
 best_score: 0.222468
 best_mode_score: 0.361181
 oot_score: 0.575197
 oot_mode_score: 0.706329
 precision@1: 0.508836
 precision@3: 0.354052
 recall@10: 0.498638
 gap_score: 0.352164

Benchmark table:

id	target	position	text	tag	gold_map	tie	predictions	best	best_m	oot	oot_m	P@1	P@3	R@10	gap
301	side	29	on sunday at craven c...	n	{'team': 5}	False	[team, outfit, ...]	1.00	1.00	1.00	1.00	1.00	0.33	1.00	1.00
302	side	2	on our side : provide...	n	{'ally': 1, ...}	True	[end, front, ...]	0.00	0.00	0.20	1.00	0.00	0.00	0.20	0.05
303	side	11	if you want to find s...	n	{'part': 4, ...}	False	[part, aspect, ...]	0.57	1.00	1.00	1.00	1.00	1.00	0.75	0.96
304	side	6	this requires some ac...	n	{'team': 2, ...}	False	[end, perspective, ...]	0.00	0.00	0.40	1.00	0.00	0.00	0.25	0.08
305	side	8	hen would that thou w...	n	{'against': 1, ...}	True	[end, bank, ...]	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
306	side	9	12/31/04 - david magd...	n	{'surface': 1, ...}	True	[hand, end, ...]	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
307	side	2	on which side shall i...	n	{'faction': 2, ...}	False	[bank, end, ...]	0.00	0.00	0.25	0.00	0.00	0.00	0.33	0.02
308	side	18	this will help the yo...	n	{'bank': 1, ...}	True	[end, bank, ...]	0.00	0.00	0.40	1.00	0.00	0.33	0.40	0.16
309	side	16	under zeldman 's lead...	n	{'aspect': 2, ...}	False	[end, half, ...]	0.00	0.00	0.60	1.00	0.00	0.33	0.50	0.15
310	side	13	gattlinburg , tennes...	n	{'boundary': 2, ...}	False	[end, bank, ...]	0.00	0.00	0.20	0.00	0.00	0.33	0.25	0.05
311	tell	24	he held obi-wan loose...	v	{'inform': 2}	False	[taught, said, ...]	0.00	0.00	1.00	1.00	0.00	0.00	1.00	0.14
312	tell	3	our engineering depar...	v	{'inform': 4}	False	[inform, say, ...]	1.00	1.00	1.00	1.00	1.00	0.33	1.00	1.00
313	tell	47	i have n't found any ...	v	{'inform': 1}	False	[warn, show, ...]	0.00	0.00	0.80	1.00	0.00	0.00	0.50	0.12
314	tell	4	" i used to tell him ...	v	{'notify': 1}	False	[say, show, ...]	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
315	tell	13	so in 1986 i sent a r...	v	{'inform': 3, 'notify': 1}	False	[informed, said, ...]	0.60	1.0.0	0.80	1.00	1.00	0.33	0.66	0.51
316	tell	11	very few people can a...	v	{'assure': 1}	False	[teach, give, ...]	0.00	0.0.0	0.00	0.00	0.00	0.00	0.00	0.00
317	tell	1	he told me , " stay w...	v	{'order': 1, ...}	True	[informed, advised, ...]	0.00	0.0.0	0.33	1.00	0.00	0.00	0.33	0.05
318	tell	5	j. reep wrote me and ...	v	{'communicate': 1, ...}	True	[gave, informed, ...]	0.00	0.0.0	0.66	1.00	0.00	0.33	0.66	0.26
319	tell	24	dear word detective i...	v	{'assure': 2, 'inform': 1}	False	[say, teach, ...]	0.00	0.0.0	1.00	1.00	0.00	0.00	1.00	0.21
320	tell	9	robi damelin took par...	v	{'recount': 1, ...}	True	[relates, describes, ...]	0.00	0.0.0	0.50	1.00	0.00	0.33	0.50	0.18
...															

B Appendix: Sample Substitution Tables

Id=865 Target=clear Position=5 Text=with the sun just barely clearing the horizon , mike switched to a split-shot rig .

candidate	is_included	candidate_prob	normalized_prob	prop_score	cls_sim	target_sim	val_score	final_score
clearing	False	0.900112	9.011174	2.198465	1.000000	1.000000	1.000000	1.021985
crossing	True	0.035565	0.356052	-1.032680	0.997828	0.900908	0.984162	0.973836
breaking	True	0.011958	0.119710	-2.122681	0.997373	0.913853	0.986092	0.964865
cleared	False	0.002018	0.020200	-3.902050	0.998433	0.956713	0.989401	0.950380
over	False	0.002366	0.023688	-3.742805	0.997864	0.824015	0.974554	0.937126
clear	False	0.038240	0.382824	-0.960180	0.998519	0.911969	0.941347	0.931745
touching	True	0.000467	0.004680	-5.364513	0.998367	0.869704	0.981233	0.927588
covering	True	0.000374	0.003744	-5.587491	0.997407	0.887289	0.978881	0.923006
below	False	0.001115	0.011162	-4.495248	0.997816	0.734785	0.963756	0.918803
passing	True	0.000141	0.001408	-6.565676	0.998772	0.877965	0.981740	0.916083
above	False	0.000374	0.003747	-5.586845	0.998001	0.754953	0.965712	0.909844
brushing	True	0.000108	0.001083	-6.828219	0.998035	0.829570	0.976807	0.908525
past	False	0.000117	0.001174	-6.747449	0.998118	0.814638	0.973937	0.906462
approaching	True	0.000063	0.000630	-7.369658	0.997882	0.854799	0.978873	0.905176
exceeding	True	0.000040	0.000399	-7.827486	0.996387	0.841635	0.975841	0.897566
rounding	True	0.000022	0.000223	-8.410485	0.997133	0.852720	0.979431	0.895327
leaving	True	0.000903	0.009045	-4.705597	0.997497	0.887466	0.937333	0.890277
setting	True	0.001873	0.018748	-3.976655	0.994238	0.824659	0.928934	0.889167
behind	False	0.000042	0.000418	-7.779543	0.998157	0.767844	0.966707	0.888912
closing	True	0.000385	0.003852	-5.559057	0.998056	0.829314	0.932096	0.876505

Figure 14 Substitutions using BERT, dropout, validation score.

Id=2000 Target=wind Position=17 Text=the history of message encryption goes back to ancient greece where messages were written on a tape wound around a stick .

candidate	is_included	candidate_prob	normalized_prob	prop_score	cls_sim	target_sim	val_score	final_score
wound	False	0.915910	10.891979	2.388027	1.000000	1.000000	1.000000	1.023880
wrapped	True	0.075860	0.902123	-0.103004	0.999998	0.960871	0.990387	0.959841
rolled	True	0.000667	0.007934	-4.836629	0.999998	0.951154	0.989019	0.902787
tied	True	0.001482	0.017622	-4.038609	0.999999	0.942945	0.990856	0.902559
wrapping	True	0.000448	0.005331	-5.234296	0.999999	0.950728	0.990952	0.898385
wrap	True	0.002532	0.030113	-3.502814	0.999997	0.932566	0.987992	0.897538
winding	False	0.000177	0.002105	-6.163448	0.999999	0.959161	0.993617	0.897526
twisted	True	0.000022	0.000263	-8.243311	0.999999	0.965470	0.994015	0.883037
held	True	0.000234	0.002785	-5.883533	0.999998	0.940174	0.988613	0.881339
woven	True	0.000016	0.000185	-8.595429	0.999999	0.957505	0.993165	0.871550
felt	True	0.000036	0.000423	-7.768742	0.999998	0.947536	0.988884	0.869849
worn	True	0.000021	0.000248	-8.301167	0.999999	0.952190	0.991691	0.869179
hooked	True	0.000015	0.000180	-8.623813	0.999999	0.953717	0.991807	0.867479
run	True	0.000019	0.000231	-8.374867	0.999997	0.946624	0.989441	0.862875
loop	True	0.000495	0.005881	-5.136060	0.999996	0.911818	0.982178	0.860458
formed	True	0.000028	0.000329	-8.020422	0.999998	0.940636	0.988996	0.860432
attached	True	0.000033	0.000393	-7.841198	0.999998	0.938747	0.989434	0.860335
stuffed	True	0.000018	0.000220	-8.422757	0.999998	0.942580	0.989836	0.858352
hung	True	0.000012	0.000144	-8.847929	0.999999	0.945510	0.988445	0.857031
coil	True	0.000101	0.001196	-6.728860	0.999997	0.922704	0.985338	0.855415

Figure 15 Substitutions using RoBERTa, dropout, target similarity.

APPENDIX B. APPENDIX: SAMPLE SUBSTITUTION TABLES

Id=1164 Target=acquire Position=4 Text=teacher education students will acquire the knowledge and skills required to apply current and emerging educational technologies both as teachers in training and as practicing teachers .

candidate	is_included	candidate_prob	normalized_prob	prop_score	cls_sim	target_sim	val_score	final_score
acquire	False	0.996680	300.174316	5.704363	1.000000	1.000000	1.000000	1.057044
obtain	True	0.001264	0.380717	-0.965700	0.999999	0.954527	0.996293	0.944870
gain	True	0.001208	0.363825	-1.011081	0.999999	0.948864	0.995194	0.938753
learn	True	0.000245	0.073780	-2.606670	0.999998	0.944425	0.993138	0.918358
develop	True	0.000335	0.100775	-2.294862	0.999998	0.921472	0.991416	0.898523
accumulate	True	0.000117	0.035146	-3.348251	0.999998	0.912479	0.991228	0.878996
acquired	False	0.000008	0.002301	-6.074512	0.999996	0.932482	0.987244	0.871737
receive	True	0.000010	0.003151	-5.760093	0.999998	0.909317	0.990809	0.851716
attain	True	0.000016	0.004759	-5.347770	0.999998	0.890804	0.989867	0.837326
acquisition	False	0.000001	0.000436	-7.737041	0.999999	0.913201	0.985443	0.835830
acquiring	False	0.000001	0.000280	-8.179589	0.999994	0.916244	0.983676	0.834448
have	False	0.000009	0.002853	-5.859289	0.999997	0.891531	0.986645	0.832938
earn	True	0.000002	0.000648	-7.341999	0.999998	0.903292	0.989576	0.829872
retain	True	0.000022	0.006546	-5.028881	0.999995	0.873540	0.984383	0.823251
ascertain	True	0.000002	0.000605	-7.409687	0.999996	0.896919	0.980023	0.822822
secure	True	0.000009	0.002589	-5.956390	0.999996	0.881940	0.983973	0.822376
gather	True	0.000002	0.000696	-7.269485	0.999998	0.894726	0.989157	0.822031
get	True	0.000002	0.000495	-7.610721	0.999997	0.898045	0.989094	0.821937
achieve	True	0.000009	0.002634	-5.939106	0.999997	0.880723	0.986528	0.821331
generate	True	0.000005	0.001431	-6.549201	0.999997	0.886092	0.982064	0.820600

Figure 16 Substitutions using RoBERTa, Duplicate Input, target similarity.

Id=1705 Target=throw Position=12 Text=for example , kareem kamel noted approvingly that : ...the prisoner swap threw into sharp relief the futility of all ' peacemaking ' efforts in the region , and the importance of resistance and steadfastness .

candidate	is_included	candidate_prob	normalized_prob	prop_score	cls_sim	target_sim	val_score	final_score
threw	False	0.655777	1.905092	0.644530	1.000000	1.000000	1.000000	1.006445
throws	False	0.089377	0.259649	-1.348424	0.999998	0.965415	0.995116	0.951931
thrown	False	0.026689	0.077533	-2.557045	0.999998	0.961885	0.993119	0.936315
put	True	0.080537	0.233968	-1.452572	0.999998	0.940881	0.990421	0.926356
brought	True	0.043889	0.127502	-2.059625	0.999997	0.943181	0.991140	0.922585
called	True	0.029154	0.084694	-2.468708	0.999998	0.947122	0.992669	0.922435
set	True	0.012753	0.037050	-3.295490	0.999998	0.942733	0.991750	0.909779
puts	True	0.016875	0.049025	-3.015434	0.999996	0.939453	0.989165	0.909299
thrust	True	0.005659	0.016439	-4.108076	0.999995	0.943307	0.988045	0.902226
throw	False	0.007223	0.020985	-3.863956	0.999995	0.935552	0.986606	0.896912
drove	True	0.000583	0.001694	-6.380899	0.999998	0.959851	0.994113	0.896042
hurled	True	0.000284	0.000824	-7.100778	0.999999	0.965458	0.995296	0.894450
cast	True	0.000608	0.001765	-6.339511	0.999999	0.957814	0.993585	0.894419
brings	True	0.005046	0.014658	-4.222742	0.999993	0.934985	0.987685	0.892757
pressed	True	0.001953	0.005674	-5.171909	0.999997	0.944152	0.991333	0.892433
tossed	True	0.000212	0.000617	-7.391253	0.999998	0.964769	0.994891	0.890857
pushed	True	0.001124	0.003266	-5.724337	0.999997	0.948055	0.991571	0.890812
drew	True	0.000495	0.001437	-6.545248	0.999998	0.954881	0.993360	0.889428
flung	True	0.000126	0.000365	-7.916731	0.999998	0.965170	0.994482	0.886003
throwing	False	0.000844	0.002451	-6.011139	0.999995	0.942775	0.986668	0.882663

Figure 17 Substitutions using RoBERTa, Dynamic Patterns, target similarity.

Id=1747 Target=vital Position=4 Text=replacing mideast oil is vital , but not by substituting equally or more vulnerable domestic sources .

candidate	is_included	probability	pll_score	probability_rank	pll_rank	rank
vital	False	0.703508	-2.886014	20.000000	20.000000	40.000000
crucial	True	0.089722	-3.050069	19.000000	18.000000	37.000000
essential	True	0.065457	-3.029971	18.000000	19.000000	37.000000
critical	True	0.008737	-3.102447	15.000000	17.000000	32.000000
imperative	True	0.013393	-3.143048	16.000000	13.000000	29.000000
necessary	True	0.005757	-3.111712	11.000000	16.000000	27.000000
important	True	0.007064	-3.130681	12.000000	15.000000	27.000000
not	False	0.007911	-3.222056	14.000000	10.000000	24.000000
paramount	True	0.003220	-3.147097	9.000000	12.000000	21.000000
vit	False	0.022442	-3.382391	17.000000	2.000000	19.000000
a	False	0.007745	-3.309171	13.000000	5.000000	18.000000
key	True	0.002176	-3.153278	7.000000	11.000000	18.000000
needed	True	0.001593	-3.141998	3.000000	14.000000	17.000000
fundamental	True	0.002224	-3.228949	8.000000	9.000000	17.000000
the	False	0.003712	-3.338988	10.000000	4.000000	14.000000
.	False	0.001808	-3.277316	5.000000	7.000000	12.000000
,	False	0.001682	-3.244632	4.000000	8.000000	12.000000
absolutely	False	0.002074	-3.379481	6.000000	3.000000	9.000000
as	False	0.001394	-3.301895	1.000000	6.000000	7.000000
very	False	0.001497	-3.406083	2.000000	1.000000	3.000000

Figure 18 Substitutions using GPT-2, beam search, sentence likelihood.

Id=505 Target=really Position=10 Text=you can sort of challenge them well , did you really know the time when you said yes ?

candidate	gpt4_rank	bart_score
actually	9.000000	-0.602156
truly	10.000000	-0.608501
honestly	8.000000	-0.714129
indeed	4.000000	-0.722967
genuinely	7.000000	-0.750942
definitely	6.000000	-0.773517
certainly	5.000000	-0.803928
sincerely	1.000000	-0.850521
authentically	3.000000	-0.864407
veritably	2.000000	-1.072544

Figure 19 Substitutions using GPT-4 combined with BARTScore.

Id=731 Target=remainder Position=19 Text=if we divide any number by 4 , we would get 1 or 2 or 3 , as the remainders .

candidate	index	is_included	target_similarity	sentence_similarity
remainders	0.000000	False	1.000000	1.000000
outcomes	7.000000	True	0.932540	0.996432
results	6.000000	True	0.925131	0.995383
residues	1.000000	True	0.893899	0.994651
byproducts	5.000000	True	0.891927	0.992742
leftovers	2.000000	True	0.916591	0.992640
residuals	4.000000	True	0.881617	0.990664
aftermath	10.000000	True	0.821438	0.988487
consequences	8.000000	True	0.867688	0.987035
effects	9.000000	True	0.620403	0.976469
remain	3.000000	True	0.429729	0.970834

Figure 20 Substitutions using LLaMa proposals and scores.